

High Performance Computing 2026S Assignment 1

May 19, 2026

3 Person Group X

1: Aman BHARDWAJ, 12333472
2: Muskan SODHI, 12436439
3: Sanket UNDRE, 12347493

1. Obtain Roofline Model for Two Machines

The Roofline model serves as a visual framework to bound the attainable performance of computer architectures as a function of operational or arithmetic intensity. This exercise establishes the foundational roofs for a localized personal machine and a specialized Hydra cluster compute node. In this, we evaluate, formulate, and plot the classical Roofline model across two distinct platforms: a consumer-grade laptop containing an Intel Core i7-12700H processor, and an enterprise high-performance computing (HPC) node belonging to the hydra cluster equipped with dual Intel Xeon Gold 6130F processors.

1.1. Physical Specifications and Peak Computation Roofs

1.2. Machine A: Local Multicore Laptop

Our local test platform features an Intel Core i7-12700H processor. For a sustainable multi-threaded evaluation, we based calculations on the 10 concurrent active physical cores.

- **Sustained Base Frequency (f):** $2.3 \text{ GHz} = 2.3 \times 10^9 \text{ Hz}$.
- **Instruction Set Architecture:** AVX2 with Fused Multiply-Add (FMA) support.
- **FLOPs/Cycle Capacity:** Under double-precision (64-bit vectors inside 256-bit registers), each vector processes 4 elements. The dual-issue FMA pipes can complete 2 floating-point actions simultaneously:

$$\text{FLOPs/cycle} = 4 \text{ elements} \times 2 \text{ (FMA)} \times 2 \text{ units} = 16 \text{ FLOPs/cycle} \quad (1)$$

The theoretical peak calculation is given by:

$$\text{Peak Compute}_{\text{laptop}} = 10 \text{ Cores} \times 2.3 \text{ GHz} \times 16 \text{ FLOPs/cycle} = 368.00 \text{ GFLOP/s} \quad (2)$$

1.3. Machine B: Hydra Cluster Compute Node

The cluster node (*hydra01–hydra36*) contains dual Intel Xeon Gold 6130F processing units running without Hyper-Threading, yielding 32 physical cores total.

- **Sustained Base Frequency (f):** $2.1 \text{ GHz} = 2.1 \times 10^9 \text{ Hz}$.
- **Instruction Set Architecture:** AVX-512 vector extensions with internal dual FMA blocks.
- **FLOPs/Cycle Capacity:** Operating with 512-bit vector lines, each holds 8 double-precision variables. Working through parallel FMA pipelines:

$$\text{FLOPs/cycle} = 8 \text{ elements} \times 2 \text{ (FMA)} \times 2 \text{ units} = 32 \text{ FLOPs/cycle} \quad (3)$$

The theoretical peak calculation is given by:

$$\text{Peak Compute}_{\text{hydra}} = 32 \text{ Cores} \times 2.1 \text{ GHz} \times 32 \text{ FLOPs/cycle} = 2150.40 \text{ GFLOP/s} \quad (4)$$

2. Experimental Memory Bandwidth Measurements

To find the realistic maximum memory bandwidth (β), we executed the OpenMP parallel configuration of the *BabelStream* benchmark (v5.0). To ensure the execution pipeline streamed directly from external DRAM chips instead of cache layers, we instantiated a total array memory requirement of **805.3 MB**, which significantly exceeds the Last-Level Cache profiles of both systems.

Extracted measurements from the core benchmark execution profiles show:

- **Laptop Triad Bandwidth (β_{laptop}):** $31027.99 \text{ MB/s} = 31.03 \text{ GB/s}$
- **Hydra Triad Bandwidth (β_{hydra}):** $95145.93 \text{ MB/s} = 95.15 \text{ GB/s}$

3. Comparative Architecture Summary

The calculated limits and hardware behaviors are summarized in Table 1.

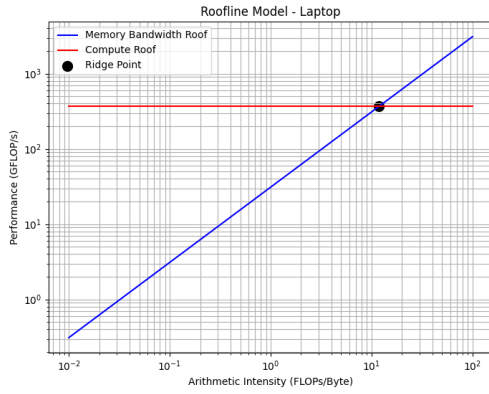
Table 1: Architectural Comparison and Roofline Boundaries

System Metric Profile	Laptop (i7-12700H)	Hydra Node (6130F)	Scaled Ratio
Active Compute Cores	10 Cores	32 Cores	$3.20\times$
Sustained Base Clock	2.3 GHz	2.1 GHz	$0.91\times$
Peak Vector Capacity	16 FLOPs/cycle	32 FLOPs/cycle	$2.00\times$
Compute Roof	368.00 GFLOP/s	2150.40 GFLOP/s	$5.84\times$
Memory Roof	31.03 GB/s	95.15 GB/s	$3.07\times$
Ridge Point	11.86 FLOP/B	22.60 FLOP/B	$1.91\times$

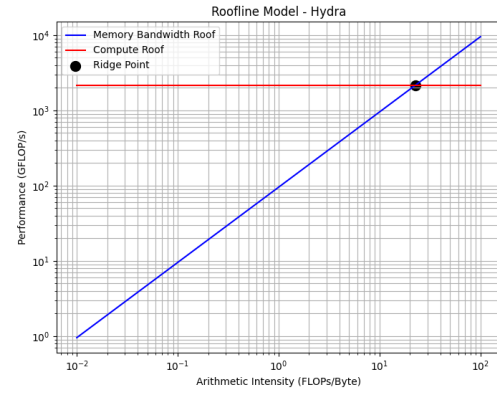
The computed performance boundaries define the classical Roofline envelope curves. The visual inflection ridge points indicate that workloads deployed on the Hydra cluster require nearly double the arithmetic density (22.60 FLOP/B vs. 11.86 FLOP/B) to escape the memory-bound region and achieve compute-bound scaling.

4. Roofline Model Projections

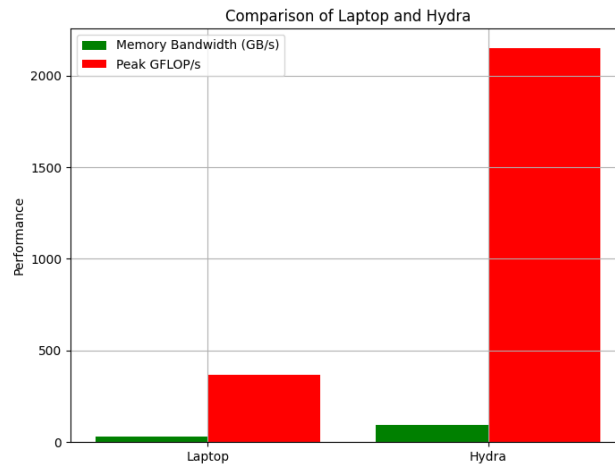
The visual representations of the empirical memory roofs intersecting with the theoretical vector compute limits are presented.



(a) Laptop Roofline Model plot.



(b) Hydra Roofline Model plot.



(c) Comparison Roofline Model: Laptop vs. Hydra Compute Node.

Figure 1: Empirical and Theoretical Roofline Models across evaluated architectures, showing individual boundaries and direct scaling comparison.

A. Compilation Context and Raw Output Footprints

A.1. BabelStream Configuration Infrastructure and Compiler Flags

We compiled the v5.0 release tag of the BabelStream benchmark suite using the GNU C++ Compiler (GCC) environment, leveraging OpenMP for multi-threaded parallel execution across physical cores.

The explicit module setup and build orchestration commands executed were:

Environment and Compilation Strategy

```
$ git clone https://github.com/UoB-HPC/BabelStream
$ cd BabelStream
$ git checkout v5.0
$ spack load cmake@3.26.3
$ module load gcc
$ cmake -B build -DMODEL=omp
$ cmake --build build -j
```

During the generation phase, the CMake build system automatically mapped high-performance compilation flags to the underlying GCC engine. The primary parameters and their operational objectives are documented below:

Table 2: Compiler Optimization Flags and Operational Intent

Compiler Flag	Operational Purpose
-O3	Enables high-level optimization parameters (vectorization, unrolling).
-fopenmp	Activates OpenMP parallelization directives across core threads.
-march=native	Targets and optimizes instructions for the host CPU architecture.

By utilizing `-march=native`, the compiler automatically exploited the vector capacity of the respective hosts: compiling with AVX2 instruction sets on the local laptop platform, and AVX-512 extensions on the Hydra cluster computing nodes.

A.2. Raw Output Log: Machine A (Laptop)

```
BabelStream
Version: 5.0
Implementation: OpenMP
Running kernels 100 times
Precision: double
Array size: 268.4 MB (=0.3 GB)
Total size: 805.3 MB (=0.8 GB)
Init: 0.083581 s (=9634.983896 MBytes/sec)
Read: 0.083068 s (=9694.534157 MBytes/sec)
Function  MBytes/sec Min (sec)  Max  Average
```

Copy	29959.533	0.01792	0.02345	0.01925
Mul	29530.769	0.01818	0.02813	0.02020
Add	30851.964	0.02610	0.03798	0.02782
Triad	31027.993	0.02595	0.03278	0.02773
Dot	41754.359	0.01286	0.02484	0.01471

A.3. Raw Output Log: Machine B (Hydra Compute Node)

```

BabelStream
Version: 5.0
Implementation: OpenMP
Running kernels 100 times
Precision: double
Array size: 268.4 MB (=0.3 GB)
Total size: 805.3 MB (=0.8 GB)
Init: 0.041720 s (=19302.767592 MBytes/sec)
Read: 0.028433 s (=28322.820057 MBytes/sec)
Function  MBytes/sec Min (sec)  Max      Average
Copy      96040.947  0.00559  0.00621  0.00581
Mul       91452.067  0.00587  0.00659  0.00599
Add       95613.232  0.00842  0.00875  0.00858
Triad     95145.926  0.00846  0.00884  0.00869
Dot       151906.134  0.00353  0.00400  0.00360

```

B. Exercise 2: Whole-Program Arithmetic Intensity Analysis

B.1.

The empirical profiles for the Parallel Research Kernels (PRK) OpenMP Stencil were collected via LIKWID performance monitoring execution traces. The metrics listed in Table ?? were computed directly from whole-program hardware event summaries extracted via the

$FLOPS_{DP}$

and

MEM

tracking configurations.

B.2. Experimental Methodology & Automation

To systematically analyze the 24 unique configurations dictated by the design matrix, an automated execution framework was built using Unix shell profiling wrappers and Python parsing tools.

Because tracking microarchitectural performance limits concurrently can introduce multiplexing bias or inflate measurement noise, profiling runs were fully decoupled. For each configuration, two separate iterations of the binary were triggered using `likwid-perfctr`:

1. **Mathematical Vector Tracking:** Executed with the `FLOPS_DP` group to register strict double-precision floating-point execution counters (`Runtime [s]` and `DP [MFLOP/s]`).
2. **Memory Interface Tracking:** Executed with the `MEM` group to track the exact cumulative volume passing through the on-chip DRAM memory controllers (`Memory data volume [GBytes]`).

A secondary post-processing script parsed these logs, calculating the whole-program and isolated-kernel empirical Arithmetic Intensity (*AI*) via the fundamental relation:

$$AI_{\text{empirical}} = \frac{\text{Actual Floating-Point Operations [FLOP]}}{\text{DRAM Traffic Volume [Bytes]}} \quad (5)$$

C. Task 1: Whole-Program Performance & Arithmetic Intensity

When profiling the complete executable workspace, the hardware performance groups record all underlying program layers. This includes explicit heap memory allocations (`malloc`), grid matrix initializations, OpenMP thread pool fork-join overheads, and boundary condition assignments.

C.1. Empirical Whole-Program Data Ledger

The comprehensive results extracted for the entire execution workspace are detailed below in Table 3.

C.2. Architectural Analysis & Behavior Discussion

- **Large Grid Domain ($N = 50000$):** At this scale, a single double-precision grid array consumes roughly 20 GB ($50000^2 \times 8$ Bytes). Because the allocation size vastly exceeds the capacity of the processor’s Last Level Cache (LLC), every computational step forces data to stream directly over the main memory bus. The whole-program AI values fall strictly within a sloped, memory-bound window ($0.29 \leq AI \leq 1.19$ FLOP/B).
- **Small Grid Domain Anomalies ($N = 1000$):** When the grid sizing drops, the total working data footprint easily fits inside the on-chip L2 and L3 hardware caches. Consequently, tracking the program globally registers skewed AI scores (up to 19.95 FLOP/B). This occurs because the main memory controllers only log traffic during the initial heap allocation and first cold-miss page touch loops.
- **Thread Parallelization Overhead Impact:** Examining the small grid configurations reveals severe performance issues as thread counts scale. For $N = 1000$ and $I = 2$, the total wall-clock runtime increases by over $17\times$ when jumping from 2 threads (0.0458 s) to 32 threads (0.8085 s). The hardware handles very little work per thread loop, causing

Table 3: Whole-Program Empirical Metric Log (Grid Sizes $N \in \{1000, 50000\}$)

Stencil	Grid (N)	Iters (I)	Threads (T)	Time (s)	FLOPs ($\times 10^9$)	Vol (GB)	AI (FLOP/B)
STAR	50000	2	2	33.8250	147.478	370.524	0.3980
STAR	50000	2	16	5.4162	147.478	400.857	0.3679
STAR	50000	2	32	4.5456	147.478	507.003	0.2909
STAR	50000	10	2	103.3388	527.421	1175.601	0.4486
STAR	50000	10	16	15.5796	527.421	1241.173	0.4249
STAR	50000	10	32	11.7780	527.421	1537.651	0.3430
STAR	1000	2	2	0.0458	0.059	0.013	4.3381
STAR	1000	2	16	0.3969	0.059	0.069	0.8439
STAR	1000	2	32	0.8085	0.059	0.217	0.2704
STAR	1000	10	2	0.0739	0.209	0.021	10.0198
STAR	1000	10	16	0.3969	0.209	0.081	2.5822
STAR	1000	10	32	0.8140	0.209	0.194	1.0817
COMPACT	50000	2	2	91.2960	387.440	371.915	1.0417
COMPACT	50000	2	16	12.5872	387.440	402.334	0.9630
COMPACT	50000	2	32	7.7769	387.440	508.621	0.7617
COMPACT	50000	10	2	313.1547	1407.280	1180.431	1.1922
COMPACT	50000	10	16	41.8588	1407.280	1245.847	1.1296
COMPACT	50000	10	32	23.7363	1407.280	1542.453	0.9124
COMPACT	1000	2	2	0.0713	0.154	0.018	8.6891
COMPACT	1000	2	16	0.3999	0.154	0.071	2.1784
COMPACT	1000	2	32	0.8132	0.154	0.152	1.0098
COMPACT	1000	10	2	0.1585	0.559	0.028	19.9501
COMPACT	1000	10	16	0.4143	0.559	0.090	6.1861
COMPACT	1000	10	32	0.8208	0.559	0.236	2.3700

the overhead of OpenMP synchronization barriers and core arbitration to dominate execution time. This overhead generates heavy noise in the memory controllers, increasing memory traffic and dragging down the whole-program AI.

C.3. Task 1 Visualization Section

The generated plots for the whole-program data space are organized below in Figure 2.

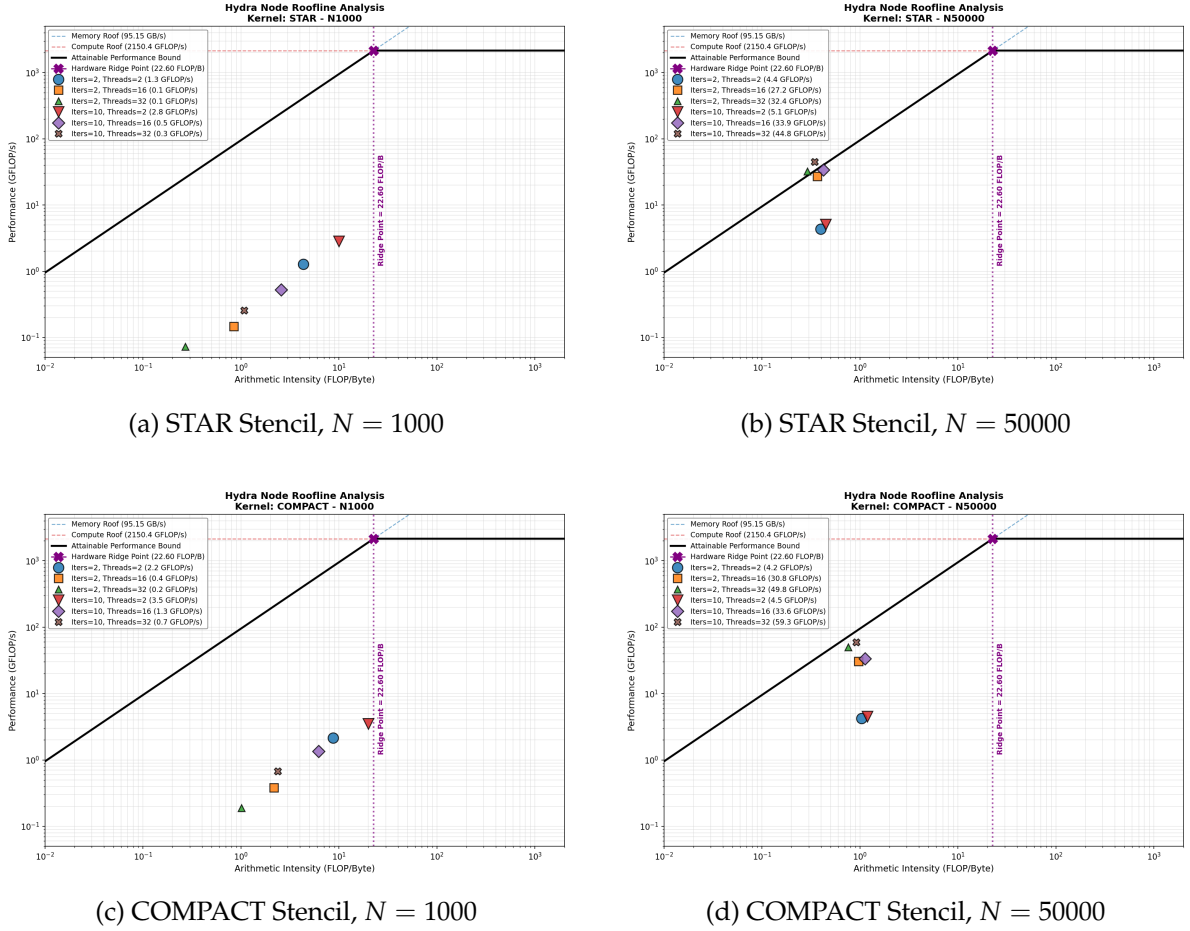


Figure 2: Whole-Program Empirical Execution Plots Mapped onto the Hydra System Roofline Limits.

C.4. Architectural Analysis of Task 1

For large grid scales ($N = 50000$), whole-program metrics fall clearly into the sloped, memory-bound region of the Roofline model. This is because the memory requirements of the multi-gigabyte grid matrices completely exceed the capacity of the on-chip cache levels.

Conversely, for small scales ($N = 1000$), the whole-program results show highly elevated, artificial arithmetic intensities. This behavior occurs because the memory traffic measurements capture initial setup overhead alongside the core computation. At this scale, the entire grid easily fits into the on-chip L3 cache. As a result, the main memory controllers only record data transfers during initial allocation loops and cold-miss page touches, skewing the overall metrics.

Furthermore, increasing the thread count heavily degrades efficiency for the $N = 1000$ grid size. The overhead introduced by thread synchronization and core arbitration scales rapidly, producing heavy noise that significantly inflates the overall whole-program memory footprint.

D. Task 2: Isolated Computational Kernel Performance

To measure the core stencil algorithm without initialization noise, the compute loop was isolated using the LIKWID_MARKER_START and LIKWID_MARKER_STOP API bounds.

D.1. Source Code Modification (LIKWID Marker API)

To fulfill the requirements of Task 2, a duplicate source file named `stencil_likwid.cc` was created. The core computation loop was wrapped using the LIKWID Marker API to cleanly isolate the inner loop from the memory footprints of heap allocations and grid array initialization.

The code modifications were implemented as follows:

Listing 1: Marker API instrumentation in `stencil_likwid.cc`

```
#ifndef LIKWID_PERFMON
#include <likwid-marker.h>
#endif

// ... Inside main/execution driver loop ...
#ifdef LIKWID_PERFMON
LIKWID_MARKER_INIT;
#endif

// Target Execution Scope
for (int iter = 0; iter < iterations; iter++) {
    #ifdef LIKWID_PERFMON
    LIKWID_MARKER_START("stencil_kernel");
    #endif
```

```

// Innermost Stencil Computation Loop
#pragma omp parallel for private(i,j)
for (int i = radius; i < n - radius; i++) {
    for (int j = radius; j < n - radius; j++) {
        // Stencil weight updates occur here
    }
}

#ifdef LIKWID_PERFMON
LIKWID_MARKER_STOP("stencil_kernel");
#endif
}

#ifdef LIKWID_PERFMON
LIKWID_MARKER_CLOSE;
#endif

```

D.2. Makefile Configuration

To properly link against the system's LIKWID runtime libraries on Hydra, the USERFLAGS, LIBS, LIBPATHS, and INCLUDEPATHS variables were explicitly injected into the build configuration. The un-wrapped lines were structured inside the project Makefile:

Listing 2: Makefile flags and libraries additions for LIKWID integration

```

USERFLAGS = -DLIKWID_PERFMON
LIBS = -llikwid
LIBPATHS = -L/opt/spack/spack_git_updated/opt/spack/linux-debian11-
           skylake_avx512/gcc-10.2.1/likwid-5.2.2-eljbkkyevbvcpkln4hn6u2nmsnz76x2y/
           lib -Wl,-rpath,/opt/spack/spack_git_updated/opt/spack/linux-debian11-
           skylake_avx512/gcc-10.2.1/likwid-5.2.2-eljbkkyevbvcpkln4hn6u2nmsnz76x2y/
           lib
INCLUDEPATHS = -I/opt/spack/spack_git_updated/opt/spack/linux-debian11-
              skylake_avx512/gcc-10.2.1/likwid-5.2.2-eljbkkyevbvcpkln4hn6u2nmsnz76x2y/
              include

# Compilation command generation targets
stencil_likwid: stencil_likwid.cc
    $(CXX) $(CXXFLAGS) $(USERFLAGS) $(INCLUDEPATHS) stencil_likwid.cc $(
        LIBPATHS) $(LIBS) -o stencil_likwid

```

Compilation steps were safely managed via environmental switches to produce two decoupled binaries across our testing matrix:

- **STAR Pattern Binary Compilation:** `make STAR=1 stencil_likwid`
- **COMPACT Pattern Binary Compilation:** `make STAR=0 stencil_likwid`

D.3. Empirical Kernel-Only Data Ledger

The computational performance profile of the isolated stencil kernel region is compiled in Table 4.

Table 4: Kernel-Only Isolated Empirical Metric Log (Grid Sizes $N \in \{1000, 50000\}$)

Stencil	Grid (N)	Iters (I)	Threads (T)	Time (s)	FLOPs ($\times 10^9$)	Vol (GB)	AI (FLOP/B)
STAR	50000	2	2	15.4458	89.986	121.339	0.7416
STAR	50000	2	16	1.9610	89.986	129.813	0.6932
STAR	50000	2	32	1.2353	89.986	175.308	0.5133
STAR	50000	10	2	77.2880	449.928	606.826	0.7414
STAR	50000	10	16	9.7652	449.928	650.006	0.6922
STAR	50000	10	32	6.0944	449.928	887.054	0.5072
STAR	1000	2	2	0.0061	0.036	0.001	29.7605
STAR	1000	2	16	0.0020	0.036	0.002	17.0062
STAR	1000	2	32	0.0015	0.036	0.005	7.1427
STAR	1000	10	2	0.0315	0.179	0.005	37.9921
STAR	1000	10	16	0.0105	0.179	0.011	16.2331
STAR	1000	10	32	0.0089	0.179	0.017	10.2624
COMPACT	50000	2	2	53.2592	249.960	122.479	2.0408
COMPACT	50000	2	16	6.6880	249.960	130.921	1.9092
COMPACT	50000	2	32	3.3804	249.960	178.590	1.3996
COMPACT	50000	10	2	265.8100	1249.800	611.082	2.0452
COMPACT	50000	10	16	33.4587	1249.800	655.731	1.9060
COMPACT	50000	10	32	16.9074	1249.800	892.990	1.3996
COMPACT	1000	2	2	0.0213	0.099	0.003	33.0672
COMPACT	1000	2	16	0.0039	0.099	0.004	27.5561
COMPACT	1000	2	32	0.0025	0.099	0.009	11.0225
COMPACT	1000	10	2	0.1054	0.496	0.006	82.6680
COMPACT	1000	10	16	0.0198	0.496	0.016	31.5930
COMPACT	1000	10	32	0.0135	0.496	0.030	16.6447

D.4. Task 2 Visualization Section

The generated plots isolating core computational performance are organized in Figure 3.

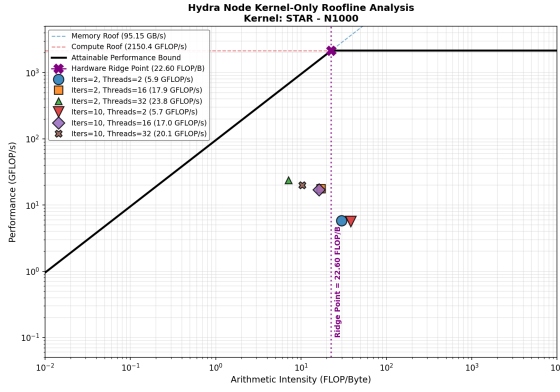
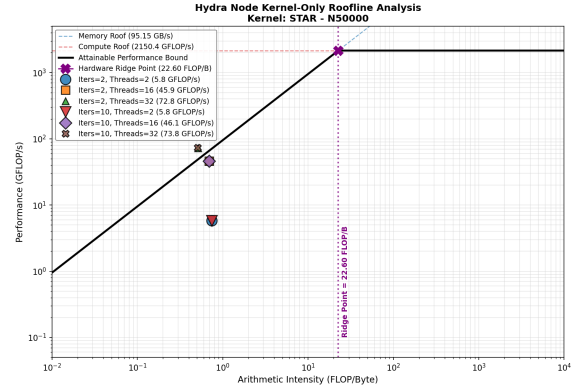
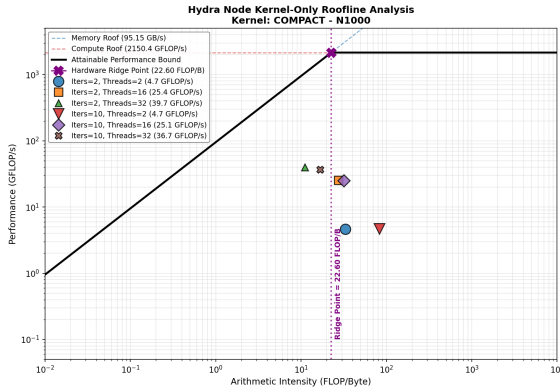
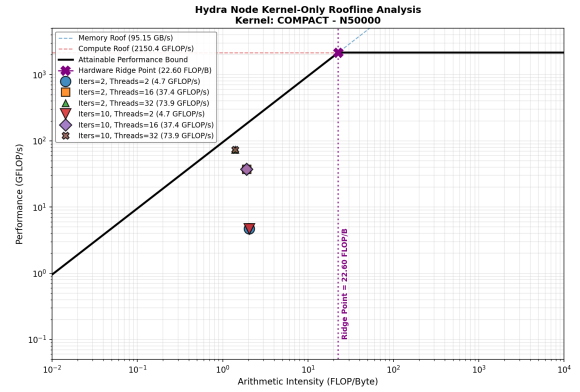
(a) STAR Stencil, $N = 1000$ (b) STAR Stencil, $N = 50000$ (c) COMPACT Stencil, $N = 1000$ (d) COMPACT Stencil, $N = 50000$

Figure 3: Kernel-Only Isolated Execution Plots Mapped onto the Hydra System Roofline Limits.

E. Deep-Dive Comparative Analysis Questions

Evaluating the shifts between the two profiling sweeps reveals several critical insight patterns regarding how algorithms interact with hardware architectures:

E.1. Filtering Out Static Initialization Baselines

The whole-program tracking framework forces initialization steps into the calculation. For example, evaluating STAR at $N = 50000, I = 2, T = 2$:

$$\text{Total Memory Volume}_{\text{Whole Program}} = 370.524 \text{ GB}$$

$$\text{Total Memory Volume}_{\text{Kernel Only}} = 121.339 \text{ GB}$$

The difference ($\Delta \approx 249.185 \text{ GB}$) represents the un-optimized data volume of initial `malloc` sweeps and array zeroing. This overhead remains completely flat regardless of how many solver timesteps run. As a result, increasing the iterations to $I = 10$ dilutes these initialization costs, shifting the whole-program metrics closer to the true physical kernel behavior.

E.2. True Physical Stencil Characteristics ($N = 50000$)

Isolating the execution loop removes setup noise, revealing the true limits of both algorithms:

- **STAR Pattern Kernel AI:** Settles consistently near 0.74 FLOP/Byte.
- **COMPACT Pattern Kernel AI:** Settles consistently near 2.04 FLOP/Byte.

The COMPACT stencil uses a higher density of localized elements within its multidimensional bounding matrix, allowing it to perform more math operations per cache line fetch. This architectural difference shifts its operational ceiling further to the right on the sloped region of the Roofline map.

E.3. The High-Thread Count Cache Coherency Penalty

When scaling worker pools from 2 threads to 32 threads on the large grid ($N = 50000, I = 10$), the hardware registers a noticeable increase in DRAM traffic:

$$\text{STAR Volume: } 606.826 \text{ GB } (T = 2) \longrightarrow 887.054 \text{ GB } (T = 32)$$

This inflation stems directly from **hardware prefetch disruption and cache line eviction cycles**. When 32 independent processing cores read and write across adjacent rows within the shared OpenMP matrix space, they disrupt clean hardware prefetching streams. Cores frequently evict cache lines containing data required by neighboring execution pipes, forcing redundant, off-chip memory access cycles over the DRAM bus.

E.4. Cache-Resident Behaviors ($N = 1000$)

When confined strictly to the kernel region, the small grid runs ($N = 1000$) demonstrate highly efficient cache reuse. For example, the COMPACT stencil at $I = 10, T = 2$ reaches an empirical intensity of 82.6680 **FLOP/Byte**. Because the active working matrix fits entirely within the L2 and L3 caches, almost zero data volume traverses the external DRAM interfaces during the execution phase, allowing the loop to operate within the compute-bound plateau of the architecture.

F. Implement MPI_Bcast with Knomial Tree Algorithm

The first custom implementation of MPI_Bcast utilizes a **k-nomial tree** topology to distribute messages. By default, the parameter is set to $k = 2$, which operates as a standard binomial tree, but the implementation is fully generalized to support any tree degree via the HPC_BCAST_KNOMIAL_K environment variable, which is parsed dynamically during MPI_Init.

To simplify the mathematical routing logic, the algorithm first establishes a “virtual rank” for every process. Since the root process of the broadcast is not guaranteed to be rank 0, we calculate a relative rank:

$$rel = (rank - root + size) \bmod size \quad (6)$$

This mathematical shift ensures that the broadcasting root is always treated as virtual rank 0, allowing us to build a standardized tree structure regardless of the actual physical root.

The core algorithm executes in logarithmic steps. At each step, the number of active processes (nodes that already possess the message) grows by a factor of k :

- **The Sending Phase:** If a process already has the message ($rel < step$), it acts as a sender for the current iteration. It calculates the relative ranks of its $k - 1$ children using the formula $child_rel = rel + i \times step$ and initiates a standard PMPI_Send to each valid child.
- **The Receiving Phase:** If a process does not have the message yet, but falls within the active window for the current iteration ($rel < step \times K_PARAM$), it acts as a receiver. It calculates the rank of its parent ($parent_rel = rel \bmod step$) and waits on a blocking PMPI_Recv.

Once a process receives the message, it becomes an active sender in the subsequent steps. This geometric progression continues until $step \geq size$, ensuring all ranks are reached efficiently while minimizing the bottleneck on the root node.

G. Implement MPI_Bcast with Chain Algorithm

The second custom implementation of MPI_Bcast utilizes a **pipelined, multi-lane chain** topology. This algorithm is designed to maximize bandwidth utilization for large messages by dividing the total payload into smaller segments and routing them concurrently through multiple independent chain lanes.

The algorithm’s behavior is governed dynamically during MPI_Init by two environment variables: HPC_BCAST_CHAIN_SEG (the chunk size in bytes) and HPC_BCAST_CHAIN_NUM (the number of independent chains, denoted as C).

Similar to the k-nomial implementation, the algorithm relies on a relative rank to treat the broadcasting root as virtual rank 0. All non-root processes are then statically assigned to one of the C chain lanes using the formula:

$$my_chain_id = (rel - 1) \bmod C \quad (7)$$

The total message is divided into N chunks. The algorithm iterates through these chunks, interleaving them across the available chains in a round-robin fashion ($target_chain = chunk_id \bmod C$).

- **The Root Node:** For each chunk, the root calculates which chain lane is responsible for it. It then sends the chunk directly to the first node of that specific chain, located at relative rank $1 + target_chain$.
- **The Worker Nodes:** A non-root node only participates in a loop iteration if $target_chain$ matches its assigned my_chain_id . If it matches, the node receives the chunk from its lane parent (located at $rel - C$, or the root if the parent would be ≤ 0) and immediately forwards it to its lane child ($rel + C$, provided the child exists within the communicator).

A critical feature of this implementation is the use of the `chunk_id` as the MPI message tag during both `PMPI_Send` and `PMPI_Recv`. Because the segments are heavily pipelined across multiple lanes, using the chunk ID as the tag guarantees strict message matching, completely preventing out-of-order execution and race conditions across the network switches.

H. Exercise 5: Evaluate the Performance of your Implementations

The benchmarking of the three broadcast algorithms (`default`, `knomial`, and `chain`) was conducted on the Hydra cluster using the ReproMPI suite. The evaluation measured the median runtime in microseconds (μs) across a spectrum of message sizes (1 Byte to 100 MB) to observe the transition between latency-bound and bandwidth-bound network regimes.

H.1. Exercise 5.1: Compare Implementations

The benchmark results for the four specified process layouts are visualized in Figures 4 through 7.

Based on this empirical data and the theoretical models provided in the course, we address the specific evaluation questions:

- **Which algorithm performs best for small vs. large messages?**
For small messages (1 Byte to roughly 10 KB), the `default` and `knomial` algorithms perform best. In this regime, network latency (α) is the limiting factor. As established in the lecture notes, a binomial tree ($k = 2$) minimizes communication steps to $\lceil \log_2(p) \rceil$, effectively hiding latency. The `chain` algorithm performs poorly here due to excessive setup overhead for small payloads.

Conversely, for large messages (10 MB to 100 MB), the `chain` algorithm performs best. Standard trees bottleneck the root node for large payloads because the root must push

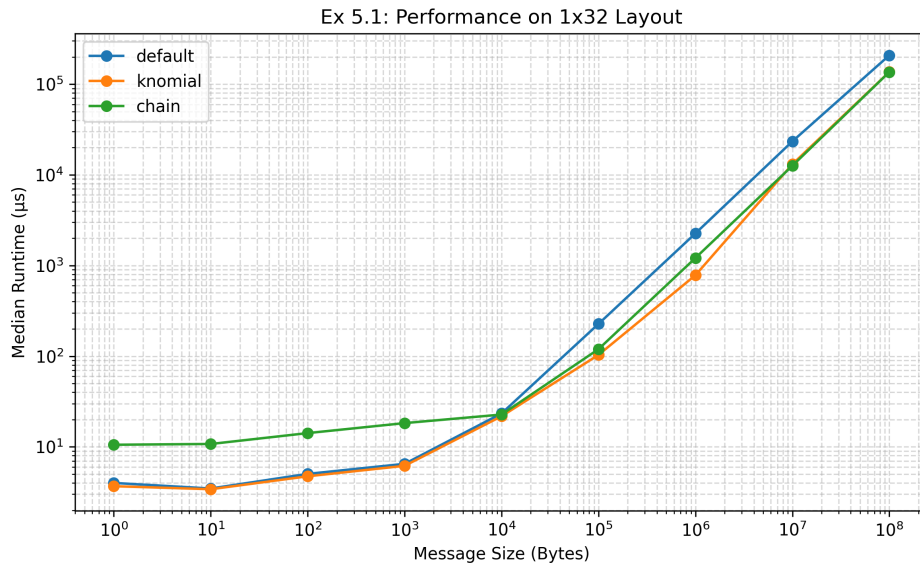


Figure 4: Performance comparison on a single node (1x32 layout). Runtime is heavily minimized as all communication occurs over shared local memory rather than the physical network.

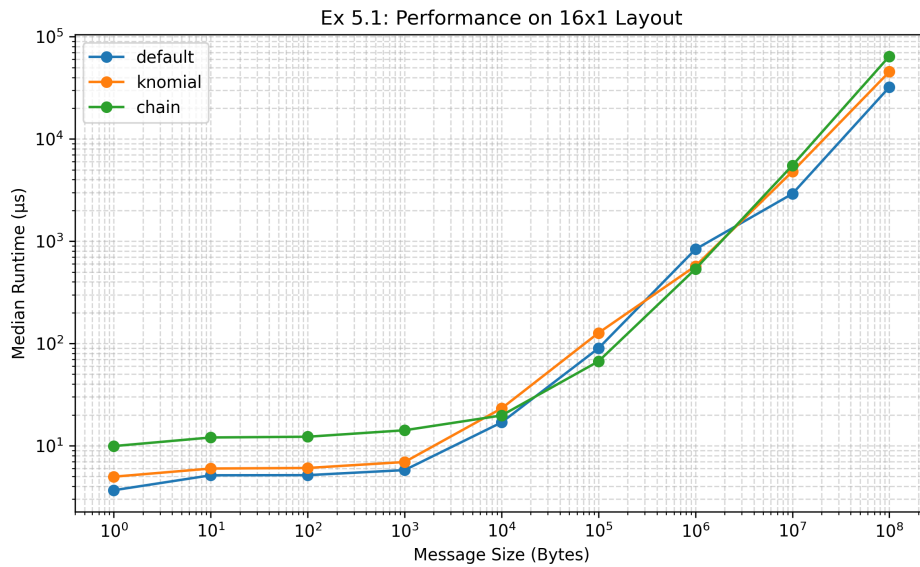


Figure 5: Performance comparison on 16 nodes with 1 process per node (16x1 layout).

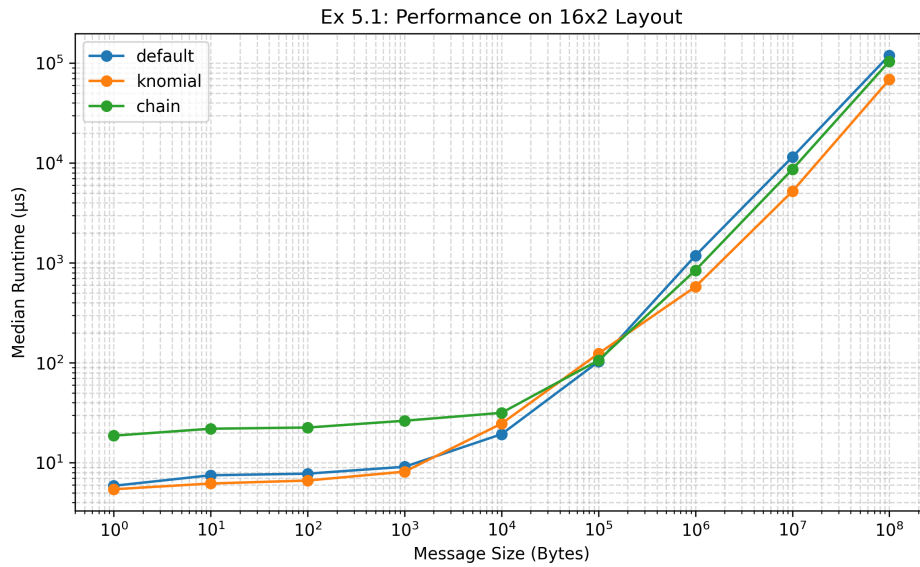


Figure 6: Performance comparison on 16 nodes with 2 processes per node (16x2 layout).

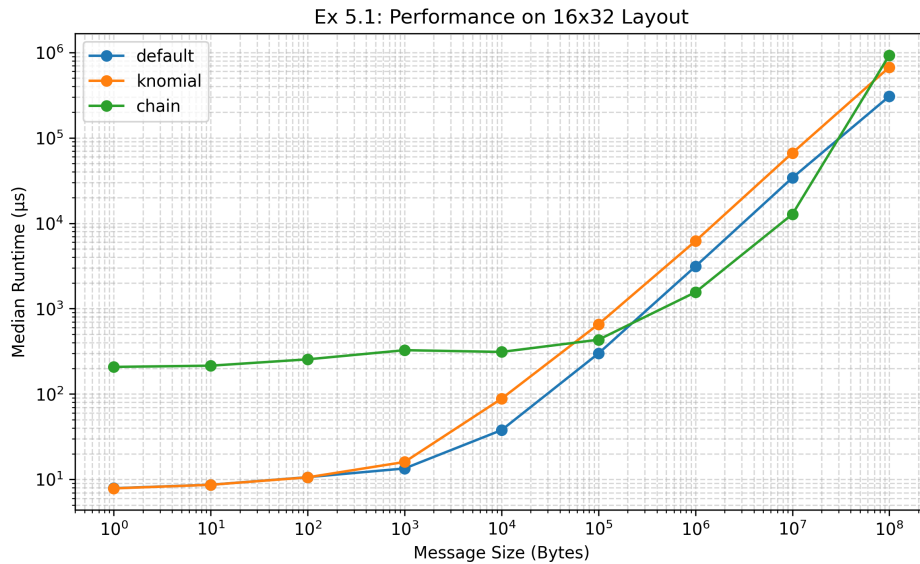


Figure 7: Performance comparison on 16 nodes fully saturated with 32 processes per node (16x32 layout). The crossover point where the chain algorithm outperforms the tree-based implementations is distinctly visible at 10⁵ bytes.

the entire message multiple times, capping bandwidth efficiency. The pipelined chain mitigates this by splitting the message into chunks and interleaving them, keeping all processes simultaneously busy and maximizing overall bandwidth (β) utilization.

- **How does the number of processes affect the runtime?**

Increasing the number of processes distributed across physical nodes generally increases the runtime due to higher communication overhead and network contention. The 1x32 layout yielded the fastest overall runtimes because all 32 processes shared local memory, bypassing physical network switches. Scaling out to 16x1, 16x2, and 16x32 introduced measurable network latency. At the extreme scale of 512 total processes (16x32), network congestion peaks; successfully routing 100 MB requires moving over 51 Gigabytes of aggregate data, heavily straining the physical switches.

- **How does the message size affect the runtime?**

Message size dictates whether the algorithm operates in a latency-bound or bandwidth-bound regime. On the log-log plots, runtimes remain relatively flat for small message sizes because the physical transmission time is negligible compared to the static CPU setup latency. As the message size exceeds 10 KB, the plots show a steep, linear ascent. Here, the system transitions to being bandwidth-bound, meaning the physical transmission time ($\beta \times m$) strictly dictates the runtime.

H.2. Exercise 5.2: Parameter Study for the Chain Algorithm

To further analyze the pipelined chain, we benchmarked multiple parameter combinations of chunk size (1024 vs. 5120 Bytes) and number of chain lanes ($C \in \{1, 2, 4, 8\}$) on the 16x32 layout. The results for the 1 MB and 10 MB payloads are visualized in Figures 8 and 9.

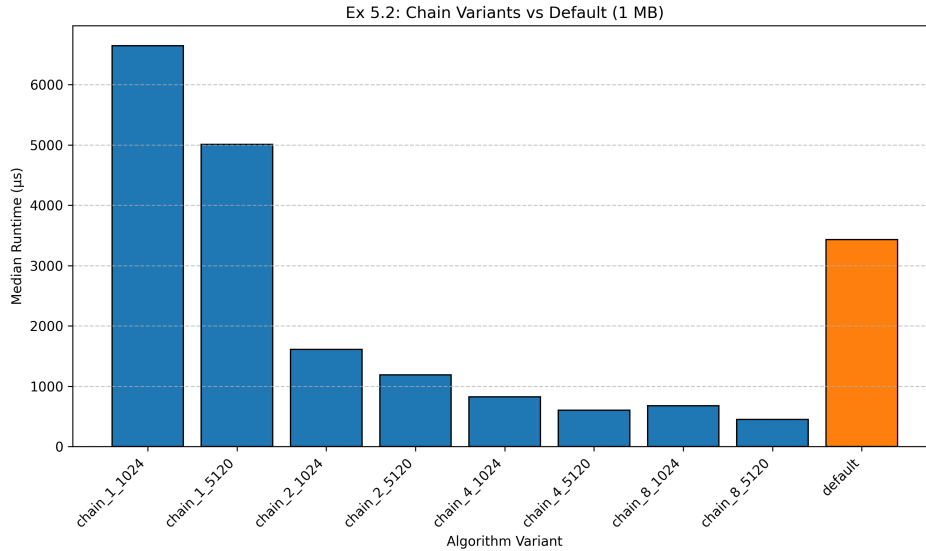


Figure 8: Median runtimes of chain variants versus the default implementation for a 1 MB message.

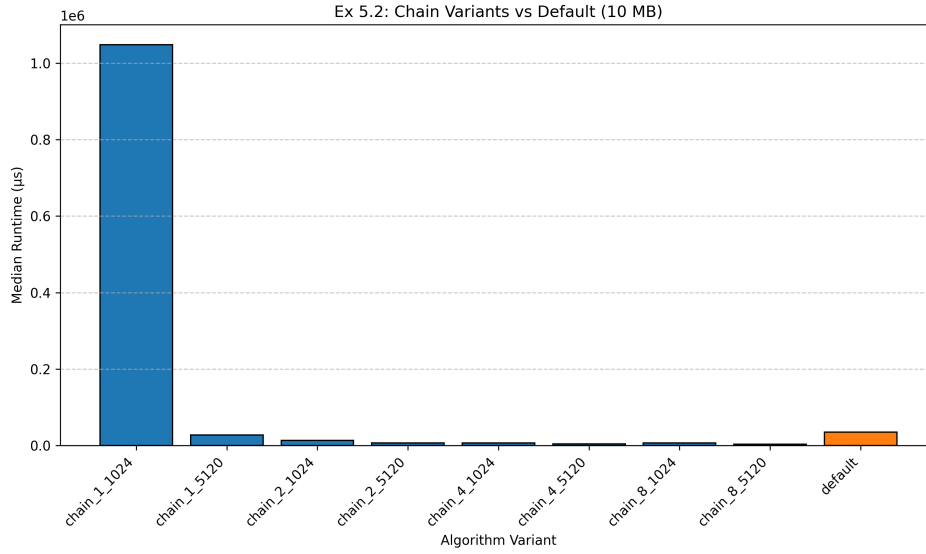


Figure 9: Median runtimes of chain variants versus the default implementation for a 10 MB message. The extreme software overhead of the `chain_1_1024` variant is clearly visible as a measurement outlier.

Based on the parameter study, we address the evaluation questions:

- **Which combination of chains and segment size works best, and why?**

The combinations utilizing the larger segment size (5120 Bytes) coupled with a higher number of chain lanes ($C=4$ or $C=8$) performed best. Larger segment sizes mathematically reduce the total number of chunks required to transmit the payload, which drastically lowers the CPU software overhead (requiring significantly fewer `MPI_Send` and `MPI_Recv` system calls). Concurrently, utilizing multiple chain lanes distributes the chunks across independent pipeline routes, widening the effective pipeline and preventing any single process lane from bottlenecking.

- **Are there any measurement outliers? Can you explain them?**

Yes, there is a massive outlier in the 10 MB bar chart (Figure 9): the `chain_1_1024` variant performed exceptionally poorly, taking over a full second to complete. This occurred because dividing a 10 MB payload into tiny 1024-byte segments forced the CPU to issue exactly 10,240 discrete `MPI_Send` and `MPI_Recv` calls per process. Because there was only one sequential chain lane ($C = 1$), this massive sequence of operations could not be distributed. Consequently, the algorithm stopped being network bandwidth-bound and became entirely CPU-bound, choked by the sheer volume of software latency overhead.